

```
import pandas as pd
from sklearn.linear_model import Perceptron
from mlxtend.plotting import plot_decision_regions
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
X_data, y_data = make_blobs(n_samples=100, centers=2,
random_state=42)
df = pd.DataFrame(X_data, columns=['feature1', 'feature2'])
df['label'] = y_data
X = df.iloc[:, 0:2]
y = df.iloc[:, -1]
p = Perceptron()
p.fit(X, y)
print("Weights:", p.coef_)
print("Bias:", p.intercept_)
z = p.score(X, y)
print("Accuracy score is:", z)
plot_decision_regions(X.values, y.values, clf=p, legend=2)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Perceptron Decision Boundary")
plt.show()
```

```

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import Perceptron

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y_and = np.array([0,0,0,1])
y_or = np.array([0,1,1,1])
y_xor = np.array([0,1,1,0])

def run_perceptron(X, y, title):
    model = Perceptron(max_iter=1000)
    model.fit(X, y)
    acc = model.score(X, y)
    print(title, "Accuracy:", acc)
    sns.scatterplot(x=X[:,0], y=X[:,1], hue=y, s=100)
    w = model.coef_[0]
    b = model.intercept_[0]
    x_vals = np.linspace(-0.5, 1.5, 100)
    if w[1] != 0:
        y_vals = -(w[0]*x_vals + b) / w[1]
        plt.plot(x_vals, y_vals, 'k--')
    plt.title(title)
    plt.show()

run_perceptron(X, y_and, "AND Gate")
run_perceptron(X, y_or, "OR Gate")
run_perceptron(X, y_xor, "XOR Gate")

```

```
from keras.models import Sequential

from keras.layers import Dense, Activation

import numpy as np

import pandas as pd

from sklearn import datasets

from sklearn.model_selection import train_test_split

from keras.utils import to_categorical

iris = datasets.load_iris()

X, y = datasets.load_iris(return_X_y=True)

y = to_categorical(y, 3)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.40)

model = Sequential()

model.add(Dense(5, input_shape=(4,)))

model.add(Activation('sigmoid'))

model.add(Dense(3))

model.add(Activation('softmax'))

model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])

model.fit(X_train, y_train, epochs=50, verbose=0)

print("\nSIGMOID RESULT")

score = model.evaluate(X_test, y_test)

print(score)

model = Sequential()

model.add(Dense(5, input_shape=(4,)))

model.add(Activation('relu'))

model.add(Dense(3))

model.add(Activation('softmax'))

model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])

model.fit(X_train, y_train, epochs=50, verbose=0)

print("\nRELU RESULT")

score = model.evaluate(X_test, y_test)

print(score)
```

```
from keras.models import Sequential
from keras.layers import Dense, Activation
from keras.utils import to_categorical
from sklearn import datasets
from sklearn.model_selection import train_test_split
X, y = datasets.load_iris(return_X_y=True)
y = to_categorical(y, 3)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.40)
model = Sequential()
model.add(Dense(8, input_shape=(4,)))
model.add(Activation('relu'))
model.add(Dense(8))
model.add(Activation('relu'))
model.add(Dense(8))
model.add(Activation('relu'))
model.add(Dense(8))
model.add(Activation('relu'))
model.add(Dense(3))
model.add(Activation('softmax'))
model.compile(loss='categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=100, verbose=0)
print("\nDEEP ANN RESULT")
score = model.evaluate(X_test, y_test)
print("Loss:", score[0])
print("Accuracy:", score[1])
```

```

import numpy as np | import pandas as pd

from keras.models import Sequential

from keras.layers import Dense, Activation

from keras.utils import to_categorical

from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train.reshape(X_train.shape[0], -1)

X_test = X_test.reshape(X_test.shape[0], -1)

X_train = X_train / 255.0

X_test = X_test / 255.0

y_train = to_categorical(y_train, 10)

y_test = to_categorical(y_test, 10)

model = Sequential()

model.add(Dense(128, input_shape=(X_train.shape[1],)))

model.add(Activation('relu'))

model.add(Dense(64))

model.add(Activation('relu'))

model.add(Dense(64))

model.add(Activation('relu'))

model.add(Dense(32))

model.add(Activation('relu'))

model.add(Dense(10))

model.add(Activation('softmax'))

model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])

epochs_list = [5, 10, 20]

results = []

for e in epochs_list:

    print("\nTraining for epochs =", e)

    model.fit(X_train, y_train, epochs=e, verbose=0)

    score = model.evaluate(X_test, y_test, verbose=0)

    print("Accuracy:", score[1])

    results.append((e, score[1]))

print("\nAccuracy vs Epochs")

for r in results:

    print("Epochs:", r[0], " Accuracy:", r[1])

```

```

import numpy as np

from keras.models import Sequential

from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

from keras.utils import to_categorical

from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

if len(X_train.shape) == 3: # MNIST
    X_train = X_train.reshape(-1, 28, 28, 1)
    X_test = X_test.reshape(-1, 28, 28, 1)

X_train = X_train / 255.0
X_test = X_test / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=X_train.shape[1:]))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

epochs_list = [5, 10, 15]

results = []

for e in epochs_list:
    print("\nTraining for epochs =", e)
    model.fit(X_train, y_train, epochs=e, verbose=0)
    score = model.evaluate(X_test, y_test, verbose=0)
    print("Accuracy:", score[1])
    results.append((e, score[1]))

print("\nAccuracy vs Epochs")

for r in results:
    print("Epochs:", r[0], " Accuracy:", r[1])

```

```

import numpy as np | import time

from keras.models import Sequential

from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

from keras.utils import to_categorical

from keras.datasets import mnist, fashion_mnist, cifar10

(X_train, y_train), (X_test, y_test) = mnist.load_data()

if len(X_train.shape) == 3: # MNIST / Fashion

    X_train = X_train.reshape(-1, 28, 28, 1)

    X_test = X_test.reshape(-1, 28, 28, 1)

X_train = X_train / 255.0

X_test = X_test / 255.0

y_train = to_categorical(y_train, 10)

y_test = to_categorical(y_test, 10)

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=X_train.shape[1:]))

model.add(Conv2D(32, (3,3), activation='relu'))

model.add(MaxPooling2D(2,2))

model.add(Conv2D(64, (3,3), activation='relu'))

model.add(Conv2D(64, (3,3), activation='relu'))

model.add(MaxPooling2D(2,2))

model.add(Flatten())

model.add(Dense(128, activation='relu'))

model.add(Dense(10, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

epochs_list = [5, 10, 20]

results = []

for e in epochs_list:

    print("\nEpochs =", e)

    start = time.time()

    model.fit(X_train, y_train, epochs=e, verbose=0)

    end = time.time()

    score = model.evaluate(X_test, y_test, verbose=0)

    print("Accuracy:", score[1])

    print("Time taken:", end - start, "seconds")

    results.append((e, score[1], end-start))

print("\nEpoch vs Accuracy vs Time")

for r in results:

    print("Epoch:", r[0], " Accuracy:", r[1], " Time:", r[2])

```

```

import numpy as np

from keras.models import Sequential

from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, BatchNormalization

from keras.utils import to_categorical

from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

if len(X_train.shape) == 3: # MNIST / Fashion

    X_train = X_train.reshape(-1, 28, 28, 1)

    X_test = X_test.reshape(-1, 28, 28, 1)

X_train = X_train / 255.0

X_test = X_test / 255.0

y_train = to_categorical(y_train, 10)

y_test = to_categorical(y_test, 10)

model = Sequential()

model.add(Conv2D(32, (3,3), padding='same', activation='relu',

                input_shape=X_train.shape[1:]))

model.add(BatchNormalization())

model.add(MaxPooling2D(2,2))

model.add(Conv2D(64, (3,3), padding='same', activation='relu'))

model.add(BatchNormalization())

model.add(MaxPooling2D(2,2))

model.add(Conv2D(64, (3,3), padding='same', activation='relu'))

model.add(BatchNormalization())

model.add(Flatten())

model.add(Dense(64, activation='relu'))

model.add(Dense(10, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

epochs_list = [5, 10, 15]

results = []

for e in epochs_list:

    print("\nEpochs =", e)

    model.fit(X_train, y_train, epochs=e, verbose=0)

    score = model.evaluate(X_test, y_test, verbose=0)

    print("Accuracy:", score[1])

    results.append((e, score[1]))

print("\nEpoch vs Accuracy")

for r in results:

    print("Epoch:", r[0], " Accuracy:", r[1])

```



```

import numpy as np | from keras.models import Sequential

from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

from keras.regularizers import l1, l2

from keras.datasets import fashion_mnist

from keras.utils import to_categorical

(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

X_train = X_train.reshape(-1,28,28,1)/255.0

X_test = X_test.reshape(-1,28,28,1)/255.0

y_train = to_categorical(y_train,10)

y_test = to_categorical(y_test,10)

def build_model(reg=None, use_dropout=False):

    model = Sequential()

    model.add(Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1),kernel_regularizer=reg))

    model.add(Conv2D(32,(3,3),activation='relu',kernel_regularizer=reg))

    model.add(MaxPooling2D(2,2))

    model.add(Conv2D(64,(3,3),activation='relu',kernel_regularizer=reg))

    model.add(Conv2D(64,(3,3),activation='relu',kernel_regularizer=reg))

    model.add(MaxPooling2D(2,2))

    model.add(Flatten())

    model.add(Dense(128,activation='relu',kernel_regularizer=reg))

    if use_dropout:

        model.add(Dropout(0.5))

    model.add(Dense(10,activation='softmax'))

    model.compile(loss='categorical_crossentropy',optimizer='adam',metrics=['accuracy'])

    return model

def train_model(name, reg=None, dropout=False):

    print("\n---", name, "---")

    model = build_model(reg, dropout)

    history = model.fit(X_train, y_train, epochs=5, verbose=0)

    train_acc = history.history['accuracy'][-1]

    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

    print("Training Accuracy:", train_acc)

    print("Test Accuracy:", test_acc)

train_model("Base Model")

train_model("L1 Regularization", reg=l1(0.001))

train_model("L2 Regularization", reg=l2(0.001))

train_model("Dropout", dropout=True)

train_model("L2 + Dropout", reg=l2(0.001), dropout=True)

```

```
import numpy as np

import matplotlib.pyplot as plt

from keras.preprocessing.image import ImageDataGenerator,
load_img, img_to_array

img = load_img('sample.jpg', target_size=(128,128))
x = img_to_array(img)
x = x.reshape((1,) + x.shape)

datagen =
ImageDataGenerator(rotation_range=30,width_shift_range=0.2,height_shift_range=0.2,
shear_range=0.2,zoom_range=0.2,horizontal_flip=True)

i = 0

plt.figure(figsize=(8,8))

for batch in datagen.flow(x, batch_size=1):
    plt.subplot(3,3,i+1)
    plt.imshow(batch[0].astype('uint8'))
    plt.axis('off')
    i += 1
    if i == 9:
        break

plt.show()
```

```
import numpy as np
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Embedding, LSTM, Dense
from keras.preprocessing.sequence import pad_sequences

num_words = 10000
max_len = 200

(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=num_words)

X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)

model = Sequential()

model.add(Embedding(input_dim=num_words, output_dim=128,
input_length=max_len))

model.add(LSTM(64))

model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',optimizer='adam',metrics
=['accuracy'])

model.fit(X_train, y_train, epochs=5, batch_size=64)

score = model.evaluate(X_test, y_test)

print("Test Accuracy:", score[1])
```

```
import numpy as np

from keras.datasets import imdb

from keras.models import Sequential

from keras.layers import Embedding, LSTM, Dense, Bidirectional

from keras.preprocessing.sequence import pad_sequences

num_words = 10000

max_len = 200

(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=num_words)

X_train = pad_sequences(X_train, maxlen=max_len)

X_test = pad_sequences(X_test, maxlen=max_len)

model = Sequential()

model.add(Embedding(input_dim=num_words, output_dim=128,
input_length=max_len))

model.add(Bidirectional(LSTM(64)))

model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',optimizer='adam',metrics
=['accuracy'])

model.fit(X_train, y_train, epochs=5, batch_size=64)

score = model.evaluate(X_test, y_test)

print("Test Accuracy:", score[1])
```

```

import numpy as np | import matplotlib.pyplot as plt

from keras.models import Model

from keras.layers import Input, Dense

from keras.datasets import mnist

(X_train, _), (X_test, _) = mnist.load_data()

X_train = X_train / 255.0 | X_test = X_test / 255.0

X_train = X_train.reshape(-1, 784)

X_test = X_test.reshape(-1, 784) | noise_factor = 0.5

X_train_noisy = X_train + noise_factor * np.random.normal(size=X_train.shape)

X_test_noisy = X_test + noise_factor * np.random.normal(size=X_test.shape)

X_train_noisy = np.clip(X_train_noisy, 0., 1.)

X_test_noisy = np.clip(X_test_noisy, 0., 1.)

input_img = Input(shape=(784,))

encoded = Dense(128, activation='relu')(input_img)

encoded = Dense(64, activation='relu')(encoded)

decoded = Dense(128, activation='relu')(encoded)

decoded = Dense(784, activation='sigmoid')(decoded)

autoencoder = Model(input_img, decoded)

autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

autoencoder.fit(X_train_noisy, X_train, epochs=5, batch_size=256, shuffle=True)

decoded_imgs = autoencoder.predict(X_test_noisy)

n = 5

plt.figure(figsize=(10,4))

for i in range(n):

    plt.subplot(3, n, i+1)

    plt.imshow(X_test_noisy[i].reshape(28,28), cmap='gray')

    plt.title("Noisy") | plt.axis('off')

    plt.subplot(3, n, i+1+n)

    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')

    plt.title("Denoised") | plt.axis('off')

    plt.subplot(3, n, i+1+2*n)

    plt.imshow(X_test[i].reshape(28,28), cmap='gray')

    plt.title("Original")

    plt.axis('off')

plt.show()

```